

Atividade 16 – CRUD com Postgres

Atividade – CRUD com PostgreSQL

Contexto

Utilize o banco de dados streaming_aula, criado nas aulas anteriores, contendo as tabelas plano, usuario, conteudo, visualizacao e log_auditoria.

A atividade deve ser realizada utilizando o PostgreSQL via terminal (psql).

Objetivo

Praticar as quatro operações fundamentais de CRUD:

- Create → INSERT
- Read → SELECT
- Update → UPDATE
- Delete → DELETE

Antes de executar um UPDATE ou DELETE, sempre que possível execute primeiro um SELECT com a mesma condição.

1. Cadastro e consulta de usuário

Cadastre um novo usuário com os seguintes dados:

- código: 10
- nome: Marcos
- e-mail: marcos@email.com
- cidade: Curitiba
- plano: 1

Depois:

1. consulte apenas o usuário cadastrado;
2. confirme o nome do plano utilizando um JOIN.

Resposta:

```
INSERT INTO usuario (id_usuario, nome, email, cidade, id_plano) VALUES (10, 'Marcos', 'marcos@email.com', 'Curitiba', 1);
```

- - 1. Consulta apenas o usuário cadastrado

```
SELECT * FROM usuario WHERE id_usuario = 10;
```

- - 2. Confirma o nome do plano utilizando JOIN

```
SELECT u.id_usuario, u.nome, u.email, u.cidade, p.nome_plano FROM usuario u JOIN plano p ON u.id_plano = p.id_plano WHERE u.id_usuario = 10;
```

2. Cadastro de vários conteúdos

Cadastre, em um único comando INSERT, os conteúdos:

Código	Título	Tipo	Gênero	Duração
201	Código Oculto	Filme	Suspense	110

202	Mundo Digital	Série	Ficção Científica	50
203	Cozinha em Família	Série	Comédia	40

Depois, consulte apenas esses três registros utilizando WHERE.

Resposta:

```
INSERT INTO conteudo (id_conteudo, titulo, tipo, genero, duracao_min) VALUES (201, 'Código Oculto', 'Filme', 'Suspense', 110), (202, 'Mundo Digital', 'Série', 'Ficção Científica', 50), (203, 'Cozinha em Família', 'Série', 'Comédia', 40);
```

-- Consulta os três registros

```
SELECT * FROM conteudo WHERE id_conteudo IN (201, 202, 203);
```

3. Alteração simples

O usuário Marcos mudou de cidade.

Altere sua cidade de Curitiba para Londrina.

Assim:

1. Consulte o registro antes da alteração;
2. Execute o UPDATE para atualizar a cidade;
3. Consulte novamente para confirmar a mudança.

Resposta:

-- 1. Consulta antes da alteração

```
SELECT * FROM usuario WHERE id_usuario = 10;
```

-- 2. UPDATE para atualizar a cidade

```
UPDATE usuario SET cidade = 'Londrina' WHERE id_usuario = 10;
```

-- 3. Consulta novamente para confirmar

```
SELECT * FROM usuario WHERE id_usuario = 10;
```

4. Alteração de várias colunas

O usuário de código 10:

- mudou para Maringá;
- passou a utilizar o plano Premium.

Atualize as duas informações em um único comando.

Depois, mostre:

- nome do usuário;
- cidade;
- nome do plano.

Resposta:

```
UPDATE usuario SET cidade = 'Maringá', id_plano = (SELECT id_plano FROM plano WHERE nome_plano = 'Premium') WHERE id_usuario = 10;
```

-- Exibe nome, cidade e nome do plano

```
SELECT u.nome, u.cidade, p.nome_plano FROM usuario u JOIN plano p ON u.id_plano = p.id_plano WHERE u.id_usuario = 10;
```

5. Atualização com expressão

A plataforma decidiu aumentar em 8% o valor de todos os planos com preço inferior a 50.00.

Antes de atualizar:

1. consulte os planos que serão afetados;
2. execute o UPDATE;
3. consulte novamente os valores atualizados.

Resposta:

- - 1. Consulta os planos afetados (preço inferior a 50.00)

```
SELECT * FROM plano WHERE preco_mensal < 50.00;
```

- - 2. Executa o UPDATE (aumento de 8%)

```
UPDATE plano SET preco_mensal = preco_mensal * 1.08 WHERE preco_mensal < 50.00;
```

- - 3. Consulta os valores atualizados

```
SELECT * FROM plano WHERE id_plano IN (1, 2);
```

6. Cadastro de visualização

Cadastre uma visualização para o usuário 10 do conteúdo 201.

Utilize:

- id da visualização: 20;
- data atual com CURRENT_DATE;
- minutos assistidos: 80.

Depois, consulte a visualização mostrando:

- usuário;
- conteúdo;
- data;
- minutos assistidos.

Resposta:

```
INSERT INTO visualizacao (id_visualizacao, id_usuario, id_conteudo, data_visualizacao, minutos_assistidos) VALUES (20, 10, 201, CURRENT_DATE, 80);
```

- - Consulta a visualização

```
SELECT u.nome AS usuario, c.titulo AS conteudo, v.data_visualizacao, v.minutos_assistidos FROM visualizacao v JOIN usuario u ON v.id_usuario = u.id_usuario JOIN conteudo c ON v.id_conteudo = c.id_conteudo WHERE v.id_visualizacao = 20;
```

7. Atualização com condição composta

Atualize para Drama Histórico o gênero de todos os conteúdos que:

- sejam do tipo Filme;
- estejam atualmente cadastrados com gênero Drama.

Antes do UPDATE, execute um SELECT com a mesma condição.

Resposta:

-- Consulta antes do UPDATE

```
SELECT * FROM conteudo WHERE tipo = 'Filme' AND genero = 'Drama';
```

-- UPDATE

```
UPDATE conteudo SET genero = 'Drama Histórico' WHERE tipo = 'Filme' AND genero = 'Drama';
```

8. DELETE seguro

Cadastre temporariamente o seguinte usuário:

- código: 30
- nome: Usuário Temporário
- e-mail: temporario@email.com
- cidade: Curitiba
- plano: 1

Depois:

1. Consulte o registro;
2. Exclua apenas esse usuário;
3. Execute um novo SELECT para confirmar que o registro foi removido.

Resposta:

```
INSERT INTO usuario (id_usuario, nome, email, cidade, id_plano) VALUES (30, 'Usuário Temporário', 'temporario@email.com', 'Curitiba', 1);
```

-- 1. Consulta o registro

```
SELECT * FROM usuario WHERE id_usuario = 30;
```

-- 2. Exclui o usuário

```
DELETE FROM usuario WHERE id_usuario = 30;
```

-- 3. Confirma a remoção

```
SELECT * FROM usuario WHERE id_usuario = 30;
```

9. Exclusão utilizando LIKE

Cadastre os conteúdos:

301	Teste Filme A
302	Teste Filme B
303	Conteúdo Permanente

Preencha os demais campos com valores válidos.

Em seguida:

1. utilize SELECT e LIKE para localizar os conteúdos cujo título começa com Teste;
2. exclua somente esses conteúdos;
3. confirme que Conteúdo Permanente continua cadastrado.

Resposta:

```
INSERT INTO conteudo (id_conteudo, titulo, tipo, genero, duracao_min) VALUES (301, 'Teste Filme A', 'Filme', 'Ação', 90), (302, 'Teste Filme B', 'Filme', 'Ação', 90), (303, 'Conteúdo Permanente', 'Filme', 'Ação', 90);
```

-- 1. Localiza os conteúdos cujo título começa com 'Teste'

```
SELECT * FROM conteudo WHERE titulo LIKE 'Teste%';
```

-- 2. Exclui somente estes conteúdos

```
DELETE FROM conteudo WHERE titulo LIKE 'Teste%';
```

-- 3. Confirma que Conteúdo Permanente continua cadastrado

```
SELECT * FROM conteudo WHERE id_conteudo = 303;
```

10. Testando integridade referencial

Tente cadastrar um usuário utilizando um id_plano que não existe, por exemplo 999.

Responda:

1. o PostgreSQL permitiu a operação?
2. qual mensagem de erro foi apresentada?
3. por que a operação foi bloqueada?
4. qual restrição da tabela está relacionada ao erro?

Resposta:

```
INSERT INTO usuario (id_usuario, nome, email, cidade, id_plano) VALUES (40, 'Teste', 'teste@email.com', 'Curitiba', 999);
```

1. O PostgreSQL permitiu a operação?

Resposta: Não. O PostgreSQL impede a gravação.

2. Qual mensagem de erro foi apresentada?

Resposta: ERRO: inserção ou atualização em tabela "usuario" viola restrição de chave estrangeira "fk_usuario_plano"

DETALHE: Chave (id_plano)=(999) não está presente na tabela "plano".

3. Por que a operação foi bloqueada?

Resposta: Porque a coluna id_plano aponta para uma tabela pai (plano), exigindo que o valor exista previamente.

4. Qual restrição da tabela está relacionada ao erro?

Resposta: Restrição de Chave Estrangeira (FOREIGN KEY).

11. Tentativa de exclusão com dependência

Escolha um usuário que possua pelo menos uma visualização.

Antes de excluir:

1. consulte suas visualizações;
2. tente excluir o usuário;
3. observe o resultado;
4. explique por que o PostgreSQL impede a exclusão.

Resposta:

-- 1. Consulta as visualizações do usuário 1

```
SELECT * FROM visualizacao where id_usuario = 1;
```

-- 2 e 3. Tenta excluir o usuário 1

```
DELETE FROM usuario WHERE id_usuario = 1;
```

Explicação: O PostgreSQL gera erro de violação de Chave Estrangeira (fk_visualizacao_usuario) porque a tabela visualizacao possui registros filhos vinculados a esse id_usuario.

12. Alteração de e-mail

Altere o e-mail de Carla para: carla.silva@email.com

Faça a alteração identificando o usuário pelo código, e não pelo nome.

Depois consulte apenas:

- código;
- nome;
- e-mail.

Resposta:

-- Identifica Carla (id_usuario = 3)

```
UPDATE usuario SET email = 'carla.silva@email.com' WHERE id_usuario = 3;
```

```
SELECT id_usuario, nome, email FROM usuario WHERE id_usuario = 3;
```

13. Atualização de vários registros

Todos os conteúdos do tipo Série com duração inferior a 50 minutos passarão a ter duração de 50 minutos.

Faça:

- um SELECT para identificar os registros afetados;
- o UPDATE;
- outro SELECT para conferir o resultado.

Resposta:

-- 1. SELECT inicial

```
SELECT * FROM conteudo WHERE tipo = 'Série' AND duracao_min < 50;
```

-- 2. UPDATE

```
UPDATE conteudo SET duracao_min = 50 WHERE tipo = 'Série' AND duracao_min < 50;
```

-- 3. SELECT final

```
SELECT * FROM conteudo WHERE tipo = 'Série';
```

14. Exclusão condicionada

Exclua conteúdos que satisfaçam simultaneamente:

- título começando com Teste;

- nenhuma visualização associada.

Primeiro localize os conteúdos, confirme que não possuem visualizações e só então execute o DELETE.

Resposta:

- - Localiza os conteúdos com título iniciando em 'Teste' e sem visualização

```
SELECT c.* FROM conteudo c LEFT JOIN visualizacao v ON c.id_conteudo = v.id_conteudo
WHERE c.titulo LIKE 'Teste%' AND v.id_visualizacao IS NULL;
```

- - Executa o DELETE

```
DELETE FROM conteudo WHERE titulo LIKE 'Teste%' AND id_conteudo NOT IN (SELECT
id_conteudo FROM visualizacao);
```

15. Ciclo CRUD completo

Implemente o ciclo CRUD completo para um usuário fictício.

CREATE

Cadastre:

- código: 50
- nome: Juliana
- e-mail: juliana@email.com
- cidade: Cascavel
- plano: 1

READ

Consulte o usuário e seu plano.

UPDATE

Altere:

- cidade para Curitiba
- plano para 2

READ novamente

Confira as alterações.

DELETE

Remova o usuário.

Verificação Final

Confirme que ele não existe mais.

Resposta:

- - CREATE

```
INSERT INTO usuario (id_usuario, nome, email, cidade, id_plano) VALUES (50, 'Juliana',
'juliana@email.com', 'Cascavel', 1);
```

- - READ

```
SELECT u.id_usuario, u.nome, p.nome_plano FROM usuario u JOIN plano p ON u.id_plano = p.id_plano WHERE u.id_usuario = 50;
```

- - UPDATE

```
UPDATE usuario SET cidade = 'Curitiba', id_plano = 2 WHERE id_usuario = 50;
```

- - READ novamente

```
SELECT u.id_usuario, u.nome, u.cidade, p.nome_plano FROM usuario u JOIN plano p ON u.id_plano = p.id_plano WHERE u.id_usuario = 50;
```

- - DELETE

```
DELETE FROM usuario WHERE id_usuario = 50;
```

- - Verificação final

```
SELECT * FROM usuario WHERE id_usuario = 50;
```

16. Cadastro seguido de movimentação

Cadastre:

1. um novo usuário;
2. um novo conteúdo;
3. uma nova visualização relacionando os dois.

Depois, utilizando JOIN, apresente:

- nome do usuário;
- título do conteúdo;
- data;
- minutos assistidos.

Por fim, tente excluir o usuário antes de excluir sua visualização e explique o resultado.

Resposta:

- - Um novo usuário

```
INSERT INTO usuario VALUES (27, 'Gabriel', 'gabriel27@gmail.com', 'Cambira', 1);
```

- - Um novo conteúdo

```
INSERT INTO conteudo VALUES (401, 'A Noiva-Cadáver', 'Filme', 'Animação', 77);
```

- - Uma nova visualização relacionando os dois

```
INSERT INTO visualizacao VALUES (30, 27, 401, CURRENT_DATE, 77);
```

- - Consulta com JOIN

```
SELECT u.nome, c.titulo, v.data_visualizacao, v.minutos_assistidos FROM visualizacao v JOIN usuario u ON v.id_usuario = u.id_usuario JOIN conteudo c ON v.id_conteudo = c.id_conteudo WHERE v.id_visualizacao = 30;
```

- - Tentativa de exclusão

```
DELETE FROM usuario WHERE id_usuario = 27;
```

Resultado: A exclusão é bloqueada porque o registro com id_visualizacao = 30 depende diretamente do id_usuario = 27 através de uma Restrição de Chave Estrangeira.

17. Correção de dados em massa

Considere que todos os usuário cadastrados em Londrina precisam ter sua cidade corrigida para:

Londrina - PR

Execute:

1. um SELECT mostrando quais usuários serão afetados;
2. o UPDATE;
3. um SELECT final.

Resposta:

-- 1. SELECT inicial

```
SELECT * FROM usuario WHERE cidade = 'Londrina';
```

-- 2. UPDATE

```
UPDATE usuario SET cidade = 'Londrina - PR' WHERE cidade = 'Londrina';
```

-- 3. SELECT final

```
SELECT * FROM usuario WHERE cidade = 'Londrina - PR';
```

18. Exclusão em ordem correta

1. Crie um usuário e uma visualização relacionada a ela.
2. Depois remova completamente os dados desse usuário do banco.
3. A exclusão deve respeitar a integridade referencial.
4. Determine e execute a ordem correta dos comandos DELETE e explique por que essa ordem é necessária.

Resposta:

-- Criação dos registros de teste

```
INSERT INTO usuario VALUES (20, 'Maria Rita', 'mariacampana@gmail.com', 'Mandaguari', 1);
```

```
INSERT INTO visualizacao VALUES (40, 20, 101, CURRENT_DATE, 30);
```

-- 1º: Remova a dependência (registro filho na tabela visualizacao)

```
DELETE FROM visualizacao WHERE id_usuario = 20;
```

-- 2º: Remova o registro principal (registro pai na tabela usuario)

```
DELETE FROM usuario WHERE id_usuario = 20;
```

Explicação: A ordem é necessária porque a tabela visualizacao possui uma Chave Estrangeira (fk_visualizacao_usuario) que faz referência à tabela usuario. Se a linha pai foi excluída primeiro, o banco bloqueia a operação para impedir dados órfãos/inconsistentes. Ao deletar o filho primeiro, a dependência deixa de existir e a remoção do pai é liberada.